

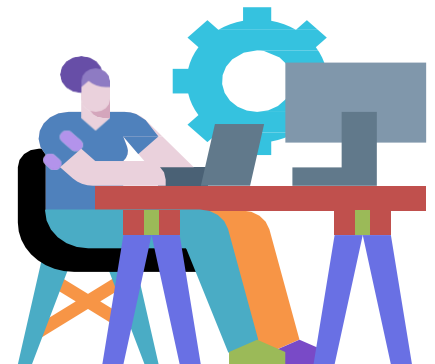
Lesson 3

Python Functions



FUNCTIONS

- Functions
 - Function overview
 - Creating a Function
 - Calling a Function
 - Default parameter Function
 - Variable-length arguments
 - Docstring
 - Return Type Function
 - Types of Functions



Functions

A function is a group of related statements that performs a specific task.

- Functions help break our program into smaller and modular chunks
- As our program grows larger and larger, functions make it more organized and manageable
- Function helps to prevent repetition and makes the code reusable
- A function is a block of code which only runs when it is called.
- You can pass data, known as parameters, into a function.
- A function can return data as a result.
- By default, a function must be called with the correct number of arguments.
- A function definitions cannot be empty, but if you for some reason have a function definition with no content, put in the pass statement to avoid getting an error.

FUNCTIONS

Creating Function

We can create a Python function using the **def** keyword.

Function Syntax

```
def function_name(parameters):  
    """docstring"""  
    statement(s)
```

function definition

```
def greet(name):  
    """  
    This function greets to  
    the person passed in as  
    a parameter  
    """  
    print("Hello, " + name + ". Good morning!")
```

Calling a Python Function

A function we can call it by using the name of the function followed by parenthesis containing parameters of that particular function.

Syntax:

Function_name (arguments) – Arguments are specified after the function name, inside the parentheses.
You can pass any amount of arguments.

Example:

Def_greet()

Scope and Lifetime of variables

Scope of a variable is the portion of a program where the variable is recognized.

```
# Program to illustrate
```

```
def my_func():  
    x = 100  
    print("Value inside function:",x)
```

```
x = 200  
my_func()  
print("Value outside function:",x)
```

Default Parameter Value

Default values indicate that the function argument will take that value if no argument value is passed during the function call. The default value is assigned by using the assignment(=) operator of the form keywordname=value.

Keyword Arguments

We can also send arguments with the key = value syntax.

```
# Program to illustrate
```

```
def student(firstname, lastname='King', colour='red'):  
    print(firstname, lastname, 'favourite colour is ', colour, 'Colour')
```

```
# 1 keyword argument  
student(firstname='James')
```

```
# 2 keyword arguments  
student(firstname='James', colour='White')
```

```
# 2 keyword arguments  
student(lastname='Gates', colour='Yellow')
```

FUNCTIONS

Arbitrary Arguments, *args

Add a * before the parameter name in the function definition if we don't know the amount of argument that will be passed into a function.

Arbitrary Keyword Arguments, **kwargs

Add a double ** before the parameter name in the function definition if we do not know how many keyword arguments

```
# Arbitrary Arguments
def name_of_colors(*kids):
    print("the colour is " + kids[1])
```

```
name_of_colors("Red", "Yellow", "Red")
```

```
#Arbitrary Keyword Arguments
def name_of_colors(**kids):
    print("the colour is " + kids["third_colour"])
```

```
name_of_colors(first_color="Red",
second_color="Yellow", third_colour="Red")
```

Docstring

This is used to describe the functionality of the function. The use of docstring in functions is optional but it is considered a good practice.

```
# Arbitrary Arguments
def greeting(name):
    """This function get the name of person and
    print greet the user"""

    print('Good doy,{}'.format(name))

name_of_colors("Joseph")
```

Syntax:
`print(functionname.__doc__)`

Example:
`print(greeting.__doc__)`

FUNCTIONS

The return statement

The return statement is used to exit a function and go back to the place from where it was called.

Syntax

Function Syntax

```
def function_name(parameters):  
    """docstring"""  
    statement(s)  
    return expression
```

Program to illustrate

```
def add_numbers(x,y):  
    """ This functions returns the sum of two  
    values"""  
    sum = x + y  
    return sum  
  
num1 = 5  
num2 = 6  
  
print("The sum is", add_numbers(num1,  
num2))
```

COMPLETE THE FOLLOWING QUESTION (15 MINS)

1. Create a function using the def keyword. The function should accept one argument. The function should check the datatype of the argument and find the absolute of the argument if the datatype is int or float. Else, it should return “I’m unable to find the absolute of this datatype”
2. Call your function and test your script with -100 and “Hello”

FUNCTIONS

Types of Functions

- Built-in functions – Functions that are built into Python
- User-defined functions – Functions defined by the users themselves

Function	Description
abs()	Returns the absolute value of a number
all()	Returns True if all items in an iterable object are true
any()	Returns True if any item in an iterable object is true
ascii()	Returns a readable version of an object. Replaces none-ascii characters with escape character
bin()	Returns the binary version of a number
bool()	Returns the boolean value of the specified object
bytearray()	Returns an array of bytes
bytes()	Returns a bytes object
callable()	Returns True if the specified object is callable, otherwise False
chr()	Returns a character from the specified Unicode code.
classmethod()	Converts a method into a class method
compile()	Returns the specified source as an object, ready to be executed
complex()	Returns a complex number
delattr()	Deletes the specified attribute (property or method) from the specified object

FUNCTIONS

Function	Description
dict()	Returns a dictionary (Array)
dir()	Returns a list of the specified object's properties and methods
divmod()	Returns the quotient and the remainder when argument1 is divided by argument2
enumerate()	Takes a collection (e.g. a tuple) and returns it as an enumerate object
eval()	Evaluates and executes an expression
exec()	Executes the specified code (or object)
filter()	Use a filter function to exclude items in an iterable object
float()	Returns a floating point number
format()	Formats a specified value
frozenset()	Returns a frozenset object
getattr()	Returns the value of the specified attribute (property or method)
globals()	Returns the current global symbol table as a dictionary
hasattr()	Returns True if the specified object has the specified attribute (property/method)
hash()	Returns the hash value of a specified object
help()	Executes the built-in help system
hex()	Converts a number into a hexadecimal value
id()	Returns the id of an object
input()	Allowing user input
int()	Returns an integer number
isinstance()	Returns True if a specified object is an instance of a specified object
issubclass()	Returns True if a specified class is a subclass of a specified object

FUNCTIONS

Function	Description
iter()	Returns an iterator object
len()	Returns the length of an object
list()	Returns a list
locals()	Returns an updated dictionary of the current local symbol table
map()	Returns the specified iterator with the specified function applied to each item
max()	Returns the largest item in an iterable
memoryview()	Returns a memory view object
min()	Returns the smallest item in an iterable
next()	Returns the next item in an iterable
object()	Returns a new object
oct()	Converts a number into an octal
open()	Opens a file and returns a file object
ord()	Convert an integer representing the Unicode of the specified character
pow()	Returns the value of x to the power of y
print()	Prints to the standard output device
property()	Gets, sets, deletes a property
range()	Returns a sequence of numbers, starting from 0 and increments by 1(by default)
repr()	Returns a readable version of an object
reversed()	Returns a reversed iterator

FUNCTIONS

Function	Description
round()	Rounds a numbers
set()	Returns a new set object
setattr()	Sets an attribute (property/method) of an object
slice()	Returns a slice object
sorted()	Returns a sorted list
staticmethod()	Converts a method into a static method
str()	Returns a string object
sum()	Sums the items of an iterator
super()	Returns an object that represents the parent class
tuple()	Returns a tuple
type()	Returns the type of an object
vars()	Returns the <code>__dict__</code> property of an object
zip()	Returns an iterator, from two or more iterators

COMPLETE THE FOLLOWING QUESTION

1. Create a python function that Implement any 10 inbuilt functions you Know.

NB: Each function must take in an argument and perform a call to one inbuilt function.

2. Create a python function that find the largest item from the below list.

Tips: Use the max() inbuilt function

`My_list = [1,8,90,23,54,9,10,4,2,500,200,234,128]`

COMPLETE THE FOLLOWING QUESTION

1. Write a function the accepts two integer parameters and returns the smallest of the two numbers.
2. Accept two input values from the user.
3. Using the function, print the smallest number

COMPLETE THE FOLLOWING QUESTION

1. Write a Python function to create and print a list the square of numbers between 1 and 50 (both included).

Expected result: [1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400, 441, 484, 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1156, 1225, 1296, 1369, 1444, 1521]

COMPLETE THE FOLLOWING QUESTION

1. Go the python console
2. Import the math inbuilt module
3. Use the dir function to get all the list of functions under the math module.
4. Implement any 5 functions.
5. Using the help function, get more detail regarding the following inbuilt modules
 1. `__main__`
 2. `random`
 3. `statistics`
 4. `sys`
 5. `time`

PYTHON RECURSION

Recursion is the process of defining something in terms of itself. it is a process in which a function calls itself directly or indirectly.

Advantages of using recursion

- A complicated function can be split down into smaller sub-problems utilizing recursion.
- Sequence creation is simpler through recursion than utilizing any nested iteration.
- Recursive functions render the code look simple and effective.

```
def func(): <--  
    |  
    | (recursive call)  
    |  
func() ----
```

Disadvantages of using recursion

- A lot of memory and time is taken through recursive calls which makes it expensive for use.
- Recursive functions are challenging to debug.
- The reasoning behind recursion can sometimes be tough to think through.

PYTHON RECURSION

```
def my_function():  
    x = 10  
    my_function()  
  
My_function()
```

When `my_function()` executes the first time, Python creates a namespace and assigns `x` the value 10 in that namespace. Then `my_function()` calls itself recursively. The second time `my_function()` runs, the interpreter creates a second namespace and assigns 10 to `x` there as well. These two instances of the name `x` are distinct from each another and can coexist without clashing because they are in separate namespaces.

In theory, `my_function()` would go on forever, calling itself over and over without any of the calls ever returning. In practice, of course, nothing is truly forever as your computer only has so much memory, and it would run out eventually.

Python doesn't allow that to happen. The interpreter limits the maximum number of times a function can call itself recursively, and when it reaches that limit, it raises a `RecursionError`.

By default, the maximum depth of recursion is 1000. If the limit is crossed, it results in `RecursionError`.

PYTHON RECURSION

You can find out what Python's recursion limit is with a function from the sys module called `getrecursionlimit()`:

```
from sys import getrecursionlimit  
  
getrecursionlimit()
```

You can change it, too, with `setrecursionlimit()`:

```
from sys import setrecursionlimit  
  
setrecursionlimit(2000)  
getrecursionlimit()
```

```
def countdown(num):  
    print(num)  
    if num == 0:  
        return # Terminate recursion  
    else:  
        countdown(num - 1) # Recursive call  
  
countdown(5)
```

1. Our recursion ends when the number reduces to 0. This is called the base condition.
2. In the recursive call, the argument is one less than the current value of n , so each recursion moves closer to the base case.

`countdown()` doesn't check its argument for validity. If n is either a non-integer or negative, it will return a `RecursionError` exception because the base case is never reached.

Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

PYTHON RECURSION

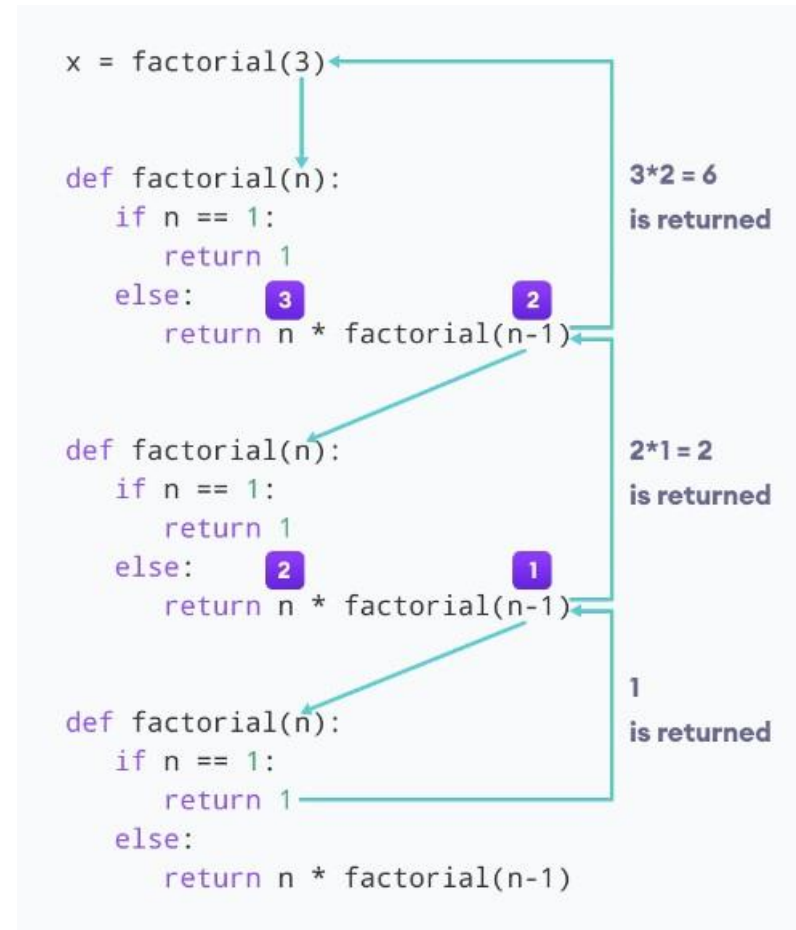
```
# Program to print factorial of a number  
# recursively.
```

```
# Recursive function
```

```
def recursive_factorial(n):  
    if n == 1:  
        return n  
    else:  
        print(n)  
        return n * recursive_factorial(n - 1)
```

```
# user input  
num = 3
```

```
# check if the input is valid or not  
if num < 0:  
    print("Invalid input ! Please enter a  
    positive number.")  
elif num == 0:  
    print("Factorial of number 0 is 1")  
else:  
    print("Factorial of number", num, "=",  
    recursive_factorial(num))
```



```
from math import factorial  
factorial(4)
```

In Python, the math module has a factorial function that provides implementation for factorial

COMPLETE THE FOLLOWING QUESTION

1. Write a recursive function that accepts two numbers as its argument and returns its power.

ANONYMOUS FUNCTIONS

An **anonymous** function is a function that is defined without a name.

While normal functions are defined using the **def** keyword. In Python, anonymous functions are defined using the **lambda** keyword.

Anonymous functions are also called lambda functions. We use lambda functions when we require a nameless function for a short period of time.

Lambda functions are used along with built-in functions like **filter()**, **map()** etc.

Syntax

```
lambda arguments: expression
```

Lambda functions can have any number of arguments but only one expression. The expression is evaluated and returned. Lambda functions can be used wherever function objects are required.

ANONYMOUS FUNCTIONS- CLASS EXAMPLE

```
def triple_function(x):  
    return x * 3  
print(triple_function(5))  
  
# Program to show the use of lambda  
functions  
triple_value = lambda x: x * 3  
print(triple_value(5))
```

In this program, `lambda x: x * 3` is the **lambda function**. Here, `x` is the **argument** and `x * 3` is the **expression** that gets evaluated and returned.

COMPLETE THE FOLLOWING QUESTION

1. Create a lambda function that adds 20 to a given number passed in as an argument
2. Create a lambda function that multiplies argument x with argument y and print the result.

ANONYMOUS FUNCTIONS WITH IF-ELSE

```
# Example of lambda function using if-else
min_value = lambda a, b : a if(a < b)
else b

print(min_value(1,2))
```

Here we are using Min lambda function to find the maximum of two integers.

ANONYMOUS FUNCTIONS - FILTER AND MAP

The **filter()** function in Python takes in a function and a list as arguments.

The function is called with all the items in the list and a **new list** is returned which contains items for which the function evaluates to True.

```
filter(lambda arguments: expression, list_name)
```

The **map()** function in Python takes in a function and a list as arguments..

The function is called with all the items in the list and a new list is returned which contains items returned by that function for each item.

```
map(lambda arguments: expression , list_name)
```

CLASS EXAMPLE

```
# Program to filter out only the even items from a list
#Filter – A new list is returned for all the items that evaluates to true.
#Map – A new list is returned for each item in the list

list_one = [1, 7, 100, 8, 9, 17, 25, 16, 35, 80, 5, 4]

new_list_one = list(map(lambda x: (x % 2 == 0), list_one))
new_list_two = list(filter(lambda x: (x % 2 == 0), list_one))

new_list_three = list(map(lambda x: x * 2, list_one))

print(new_list_one)
print(new_list_two)
print(new_list_three)
```

CLASS EXAMPLE -INTERSECTION OF TWO ARRAYS

Function to find intersection of two arrays

```
def my_interSection(arr1,arr2):
```

```
    # filter(lambda x: x in arr1, arr2) -->
```

```
    # filter element x from list arr2 where x
```

```
    # also lies in arr1
```

```
    result = list(filter(lambda x: x in arr1, arr2))
```

```
    print ("Intersection : ",result)
```

Driver program

```
if __name__ == "__main__":
```

```
    arr_one = [ 11, 13, 14, 15, 17]
```

```
    arr_two = [ 12, 13, 15, 16]
```

```
    my_interSection(arr_one,arr_two)
```


COMPLETE THE FOLLOWING QUESTION

Write a Python program to count the even, odd numbers in the below given array of integers using Lambda.

```
arry_nums = [1, 2, 3, 5, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
```

COMPLETE THE FOLLOWING QUESTION

Write a Python program to add two given lists using map and lambda.

Original list:

[11,12,13]

[14,15,16]

Expected result: after adding two list [25, 27,29]